NumPy: fast math on arrays

🎯 Today's goal

By the end of this hour you will:

- turn a list of numbers into a NumPy **array**
- do maths on **every number at once**, without a loop
- pick out values with a **mask**, like "only the days over 10 000 steps"
- work with a **2D array** (a table) and average its rows or columns
- make reproducible random numbers

🤔 Why this matters

Welcome to the **Data Science** phase! Yesterday you added up expenses one by one with a loop. That's fine for ten numbers, but real data has thousands or millions of them.

NumPy (short for "Numerical Python") is a module that stores numbers in an **array**: a grid of numbers that all have the same type. Think of the difference between a cashier scanning items one at a time (a loop) and a machine that weighs the whole trolley at once (an array). NumPy does the whole-trolley maths, and it's very fast.

Almost every data science tool is built on NumPy: pandas tables (tomorrow!), charts, and machine learning models all keep their numbers in NumPy arrays.

🔢 Your first array

Everyone imports NumPy with the nickname `np` (remember `import ... as ...` from Day 7). `np.array()` turns a list into an array. Look what `* 2` does to each of them:

```
import numpy as np

# Made-up daily step counts for one week
steps_list = [8200, 10450, 6300, 12100, 9800, 4300, 11250]
steps = np.array(steps_list)

print(steps_list * 2)     # a list: repeated, twice as long!
print(steps * 2)          # an array: every number doubled
```

```
[8200, 10450, 6300, 12100, 9800, 4300, 11250, 8200, 10450, 6300, 12100, 9800,
4300, 11250]
[16400 20900 12600 24200 19600  8600 22500]
```

With a list you'd need a loop (`[s * 2 for s in steps_list]`). With an array the maths happens on every element at once. This is called **element-wise** (or "vectorised") maths. An array prints without commas, which is how you can tell it apart from a list.

Every array knows a few things about itself:

```
print(type(steps))
print("dtype:", steps.dtype)    # the type of every number inside
print("shape:", steps.shape)    # the size: 7 numbers in one row
print("ndim:", steps.ndim)      # number of dimensions: 1 = a simple row
print("len:", len(steps))
print("first:", steps[0], " last:", steps[-1], " Mon-Wed:", steps[:3])
```

```
<class 'numpy.ndarray'>
dtype: int32
shape: (7,)
ndim: 1
len: 7
first: 8200  last: 11250  Mon-Wed: [ 8200 10450  6300]
```

Attribute	Means
`dtype`	"data type": `int` = whole numbers, `float` = decimals. The number after it (`int32` , `int64` , `float64`) is how much memory each number gets. In this browser whole numbers are `int32` ; on most computers you'll see `int64` .
`shape`	the size, written in brackets: `(7,)` means 7 numbers; `(3, 4)` means 3 rows and 4 columns
`ndim`	number of dimensions: 1 for a row of numbers, 2 for a table

Indexing and slicing work just like lists (Day 3).

NumPy can also **make** arrays for you:

```
print(np.arange(0, 10, 2))      # like range(): start, stop (not included), step
print(np.linspace(0, 1, 5))     # 5 evenly spaced numbers from 0 to 1 (both included)
print(np.zeros(4))              # four zeros
print(np.ones(3))               # three ones
```

```
[0 2 4 6 8]
[0.   0.25 0.5  0.75 1.  ]
[0. 0. 0. 0.]
[1. 1. 1.]
```

➗ Maths on every element

Any operator works element-wise: `+ - * / ** //`. With two arrays of the same length, NumPy pairs up the numbers position by position:

```
# Made-up café data: price and cups sold for 4 drinks
prices = np.array([2.5, 3.0, 3.5, 1.8])
cups = np.array([31, 45, 12, 60])

revenue = prices * cups              # 2.5*31, 3.0*45, 3.5*12, 1.8*60
print("Revenue per drink:", revenue)

# np.round rounds every number in the array
print("Steps in thousands:", np.round(steps / 1000, 1))
```

```
Revenue per drink: [ 77.5 135.   42.  108. ]
Steps in thousands: [ 8.2 10.4  6.3 12.1  9.8  4.3 11.2]
```

🎭 Comparing and filtering with masks

Comparing an array with a number also works element-wise. The answer is an array of `True` / `False`, one per element:

```
goal_reached = steps >= 10000
print(steps)
print(goal_reached)
```

```
[ 8200 10450  6300 12100  9800  4300 11250]
[False  True False  True False False  True]
```

A True/False array like this is called a **mask**. Put it inside square brackets and NumPy keeps only the elements where the mask is `True`, like holding a stencil over the data. And because `True`

counts as 1 and `False` as 0, `.sum()` of a mask **counts** the True values:

```
print("Good days:", steps[goal_reached])
print("Number of good days:", goal_reached.sum())

# The same in one line, and with two conditions (& means "and"; use brackets!)
print("Lazy days:", steps[steps < 7000])
print("Medium days:", steps[(steps >= 7000) & (steps < 10000)])
```

```
Good days: [10450 12100 11250]
Number of good days: 3
Lazy days: [6300 4300]
Medium days: [8200 9800]
```

With arrays, use `&` instead of `and` (and `|` instead of `or`), and put each comparison in brackets. You'll see the same rule in pandas.

📏 Summing up: aggregations

An **aggregation** squeezes many numbers into one: the total, the average, the biggest. Arrays have them built in. `std` is the **standard deviation**: roughly, how far the values typically are from the average (small = all similar, big = very spread out). `argmax` gives the **position** of the biggest value, not the value itself:

```
days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

print("Total:  ", steps.sum())
print("Mean:   ", steps.mean().round(1))
print("Min/Max:", steps.min(), steps.max())
print("Std:    ", steps.std().round(1))

best = steps.argmax()           # position of the biggest value
print("Best day:", days[best], "with", steps[best], "steps")
```

```
Total:   62400
Mean:    8914.3
Min/Max: 4300 12100
Std:     2601.7
Best day: Thu with 12100 steps
```

🗂️ 2D arrays: tables of numbers

A list of lists becomes a **2D array**: rows and columns, like a spreadsheet. Here each row is a student and each column is a test:

```
# Made-up scores: 3 students (rows) x 4 tests (columns)
scores = np.array([
    [72, 85, 64, 90],    # Amira
    [55, 61, 70, 58],    # Ben
    [88, 92, 79, 95],    # Chen
])

print(scores)
print("shape:", scores.shape, " ndim:", scores.ndim)
print("Ben's 3rd test:", scores[1, 2])    # [row, column], counting from 0
print("Chen's row:", scores[2])
print("Test 1 column:", scores[:, 0])     # ":" means "all rows"
```

```
[[72 85 64 90]
 [55 61 70 58]
 [88 92 79 95]]
shape: (3, 4)  ndim: 2
Ben's 3rd test: 70
Chen's row: [88 92 79 95]
Test 1 column: [72 55 88]
```

Read `scores[1, 2]` as "row 1, column 2". A colon on its own means "everything", so `scores[:, 0]` is "all rows, column 0": the whole first column. Slices work too: `scores[0:2, 1:3]` would be rows 0–1 and columns 1–2.

Aggregations on a 2D array can work along an **axis**. `axis=0` goes **down** the rows and gives one answer per column. `axis=1` goes **across** the columns and gives one answer per row:

```
print("Overall mean:", scores.mean())
print("Mean per test:   ", scores.mean(axis=0).round(1))   # axis=0: one per column
print("Mean per student:", scores.mean(axis=1).round(1))   # axis=1: one per row
print("Best score of each student:", scores.max(axis=1))
```

```
Overall mean: 75.75
Mean per test:    [71.7 79.3 71.  81. ]
Mean per student: [77.8 61.  88.5]
Best score of each student: [90 70 95]
```

A trick to remember axes: the axis you name is the one that *disappears*. `scores` has shape `(3, 4)`. `axis=0` squeezes away the 3 rows and leaves 4 numbers; `axis=1` squeezes away the 4 columns and leaves 3 numbers.

📡 Broadcasting: different shapes together

What happens with `scores + 5`? NumPy "stretches" the single number to match every element. This stretching is called **broadcasting**. It also works with a row: a list of 4 numbers is added to **each** row of a 3 × 4 table, so every column gets its own bonus:

```
print(scores + 5)                  # 5 bonus points everywhere

bonus = np.array([0, 0, 10, 0])    # test 3 was too hard: +10 on that column only
print(scores + bonus)

# Turn each test into a percentage of its maximum possible points
max_points = np.array([100, 100, 80, 100])
print(np.round(scores / max_points * 100, 1))
```

```
[[ 77  90  69  95]
 [ 60  66  75  63]
 [ 93  97  84 100]]
[[72 85 74 90]
 [55 61 80 58]
 [88 92 89 95]]
[[72.  85.  80.  90. ]
 [55.  61.  87.5 58. ]
 [88.  92.  98.8 95. ]]
```

Broadcasting only works when the shapes fit. Here is a mistake on purpose: 3 numbers and 2 numbers can't be paired up:

💡 This example has a mistake on purpose. Run it and read the error.

```
week1 = np.array([5, 7, 6])
week2 = np.array([4, 8])
week1 + week2
```

> ⚠️ **ValueError**
>
> The kind of value is right but the value itself isn't, like int("hello").
>
> ```
> Traceback (most recent call last):
>   line 3, in <module>
>     week1 + week2
>     ~~~~~~^~~~~~~
> ValueError: operands could not be broadcast together with shapes (3,) (2,)
> ```

"operands could not be broadcast together with shapes (3,) (2,)" means: the sizes don't match. When you see it, print the `.shape` of both arrays.

🎲 Random numbers with NumPy

NumPy can make whole arrays of random numbers, which is great for creating made-up data to practise on. First you make a random number **generator** with a seed (like `random.seed(42)` on Day 7), so the "random" numbers are the same every time:

```
rng = np.random.default_rng(42)

dice = rng.integers(1, 7, size=10)       # 10 whole numbers from 1 to 6
print("Dice:", dice)

# 5 made-up heights: average 170 cm, spread 8
heights = rng.normal(170, 8, size=5)
print("Heights:", np.round(heights, 1))

print("Share of sixes:", (dice == 6).mean())  # the mean of a mask = the share of True
```

```
Dice: [1 5 4 3 3 6 1 5 2 1]
Heights: [159.6 171.  167.5 169.9 163.2]
Share of sixes: 0.1
```

`rng.normal(170, 8, ...)` draws numbers from the famous bell-shaped "normal" curve: most values land near 170, fewer far away. And note the last trick: the **mean of a mask** is the share (fraction) of True values. You'll use it a lot.

⚠️ Common mistakes

- Forgetting `import numpy as np` → `NameError: name 'np' is not defined`.
- Using `and` / `or` with arrays: use `&` / `|` and brackets around each comparison.
- Mixing up the axes: `axis=0` gives one result per **column**, `axis=1` one per **row**.
- Adding arrays whose shapes don't fit → "could not be broadcast". Check `.shape`.
- Confusing `argmax()` (the position) with `max()` (the value).
- Writing a loop when a single array operation would do.

✅ Recap

- `np.array(list)` makes an array; it has a `dtype`, `shape` and `ndim`.
- `np.arange`, `np.linspace`, `np.zeros`, `np.ones` create arrays.
- Maths and comparisons work on every element at once.
- A mask (`arr > 5`) filters: `arr[arr > 5]`; `mask.sum()` counts, `mask.mean()` gives the share.
- `sum`, `mean`, `min`, `max`, `std`, `argmax`; on 2D arrays add `axis=0` (per column) or `axis=1` (per row).
- 2D indexing: `arr[row, column]`; `arr[:, 0]` is the first column.
- Broadcasting stretches a number or a row to fit; random numbers come from `np.random.default_rng(42)`.

🏋️ Exercises

Write your answer in the space under each exercise. Try it on the computer too, by opening this day's `index.html`. Solutions are in the answer key at the end.

1. Celsius to Fahrenheit

Convert all the made-up temperatures at once (no loop!) into a new array called `fahrenheit`. The formula is: °F = °C × 9 / 5 + 32.

```
import numpy as np

celsius = np.array([18.5, 22.0, 30.5, 15.0, 27.5])

# Create fahrenheit
```

Hints:

1. Do the maths on the whole array, exactly like on one number.
2. `fahrenheit = celsius * 9 / 5 + 32`

2. Hot days

Here are two weeks of made-up afternoon temperatures. Using a mask:

- store the temperatures **above 30** in an array called `hot_temps`
- store **how many** days were above 30 in a variable called `hot_count`

```
import numpy as np

temps = np.array([28, 31, 33, 29, 27, 35, 30, 32, 26, 25, 31, 34, 29, 30])

# Create hot_temps and hot_count
```

Hints:

1. The mask is `temps > 30`. "Above 30" means 30 itself doesn't count.
2. `hot_temps = temps[temps > 30]`
3. Count with `(temps > 30).sum()` or `len(hot_temps)`.

3. Weekly step stats

For this made-up week of step counts, create:

- `mean_steps` : the average number of steps
- `max_steps` : the most steps in one day
- `best_day` : the **name** of the day with the most steps (use `argmax` and the `days` list)

```
import numpy as np

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
steps = np.array([7400, 9100, 11800, 6200, 8800, 13400, 5100])

# Create mean_steps, max_steps and best_day
```

Hints:

1. `steps.mean()` and `steps.max()`
2. `steps.argmax()` gives a position, like `5`. Use it as an index into `days`.
3. `best_day = days[steps.argmax()]`

4. Averages per student and per test

The made-up table has one row per student and one column per test. Create:

- `student_avg` : the average of **each student** (one number per row)
- `test_avg` : the average of **each test** (one number per column)

```
import numpy as np

scores = np.array([
    [65, 70, 80],
    [90, 85, 95],
    [50, 62, 58],
    [77, 81, 73],
])

# Create student_avg and test_avg
```

Hints:

1. One number per row means going **across** the columns: `axis=1` .
2. `student_avg = scores.mean(axis=1)`
3. `test_avg = scores.mean(axis=0)`

5. Challenge: scale every column from 0 to 1

The made-up table holds people's height (cm) and weight (kg). The two columns use very different numbers, which confuses many machine learning models. **Min-max scaling** fixes this: in each column, the smallest value becomes 0, the biggest becomes 1, and the rest fall in between:

`scaled = (value − column minimum) / (column maximum − column minimum)`

Create `scaled` for the whole table at once, using `axis=0` and broadcasting.

```
import numpy as np

people = np.array([
    [160, 55],
    [175, 72],
    [182, 90],
    [168, 61],
    [190, 85],
])

# Create scaled
```

Hints:

1. `people.min(axis=0)` gives the minimum of each column: `[160 55]`.
2. Store `low = people.min(axis=0)` and `high = people.max(axis=0)`. Broadcasting subtracts them from every row.
3. `scaled = (people - low) / (high - low)`

➡️ Tomorrow

Pandas I: loading and inspecting data. NumPy arrays hold numbers, but real tables also have column names, text and dates. Tomorrow you'll meet pandas, the most popular data tool in Python: you'll load a made-up CSV file into a **DataFrame**, look at its first rows, get quick statistics with one command, pick columns and add new ones.

🔑 Answer key

Try the exercises yourself first! There is often more than one correct answer.

1. Celsius to Fahrenheit

```
import numpy as np

celsius = np.array([18.5, 22.0, 30.5, 15.0, 27.5])

fahrenheit = celsius * 9 / 5 + 32
```

2. Hot days

```
import numpy as np

temps = np.array([28, 31, 33, 29, 27, 35, 30, 32, 26, 25, 31, 34, 29, 30])

hot_temps = temps[temps > 30]
hot_count = (temps > 30).sum()
```

3. Weekly step stats

```
import numpy as np

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]
steps = np.array([7400, 9100, 11800, 6200, 8800, 13400, 5100])

mean_steps = steps.mean()
max_steps = steps.max()
best_day = days[steps.argmax()]
```

4. Averages per student and per test

```
import numpy as np

scores = np.array([
    [65, 70, 80],
    [90, 85, 95],
    [50, 62, 58],
    [77, 81, 73],
])

student_avg = scores.mean(axis=1)
test_avg = scores.mean(axis=0)
```

5. Challenge: scale every column from 0 to 1

```
import numpy as np

people = np.array([
    [160, 55],
    [175, 72],
    [182, 90],
    [168, 61],
    [190, 85],
])

low = people.min(axis=0)
high = people.max(axis=0)
scaled = (people - low) / (high - low)
```